1. Criar a classe Equacao

Equacao
+ a: int + b: int + c: int + delta: int
+ calcDelta(): void + raiz(): String

Classe: Equacao	
Métodos	**calcDelta()**: Calcula o valor de Δ através da fórmula **b² - 4 * a* c** e insere o resultado no atributo delta. **raiz()**: Exibe o nº de raízes existentes na equação através das seguintes condições: ▪ Δ < 0 → não existe raiz real. ▪ Δ = 0 → existe uma raiz real. ▪ Δ > 0 → existem duas raízes reais.

Crie a classe TesteEquacao:

Clique com o botão direito em src, acesse New --> Class e coloque o nome TesteEquacao, em seguida crie o método public static void main(String args[]).

Instancie um objeto do tipo Equacao chamado eq. Altere os atributos a, b e c com os valores digitados pelo usuário.

Exiba uma mensagem com o valor do delta e a quantidade de raízes.

Exercício 2.: Sistema de Consumo de Energia

Crie uma classe chamada **`Eletrodomestico`** que ajuda a calcular o gasto mensal de um aparelho, conforme diagrama abaixo:

Eletrodomestico
+ nome: String + potenciaWatts: double + horasPorDia: double
+ calcularConsumoDiario(): double + calcularCustoMensal(valorKwh: double): double + exibirDados(): void

Métodos

- **`calcularConsumoDiario`**(): Não recebe parâmetro. Deve retornar o consumo em kWh (Fórmula: $(potenciaWatts \times horasUsoPorDia) / 1000$).

- **`calcularCustoMensal(double valorKwh):`** Recebe como parâmetro o preço do kWh da sua região. Deve retornar o valor em reais gasto em 30 dias (Fórmula: $consumoDiario \times 30 \times valorKwh$).

- **`exibirDados():`** Método void que imprime o nome do aparelho e o consumo diário calculado.

Exercício 3.: Gestão de Conta bancária simples

Crie uma classe chamada **`ContaBancaria`** para simular operações básicas, conforme diagrama abaixo:

ContaBancaria
+ titular: String + saldo: double
+ depositar(valor: double): void + sacar(valor: double): void + verificarStatus(): String

Métodos

- `depoistar`(*valor*: *double*): Recebe um valor e soma ao saldo.

- `sacar(valor: double):` Recebe um valor e, se o saldo for suficiente, subtrai do saldo. Caso contrário, exibe uma mensagem: "Saldo insuficiente".

- `verificaStatus():` Retorna um String. Se o saldo for maior que zero, retorna "Conta Positiva", se for exatamente zero, "Conta Zerada". Se for menor que zero, "Conta em Débito".

`Crie a classe TesteConta, instancie um objeto e simule 2 depósitos e 3 tentativas de saque.`